

Resumen SQL/DES

1 DDL - Definición de Datos

CREATE TABLE: Crea una nueva tabla.

```
1 create table tabla(  
2     col1 tipo primary key,  
3     col2 tipo,  
4     col3 tipo  
5 );
```

Tipos: string, int, float, etc.

PRIMARY KEY: Define clave primaria (simple o compuesta).

2 DML - Manipulación de Datos

INSERT: Inserta tuplas en una tabla.

```
1 insert into tabla  
2     values('val1','val2',123);  
3 insert into tabla(col1,col2)  
4     values('val1','val2');
```

NULL: Representa valores desconocidos/no aplicables.

3 Consultas SELECT

SELECT básico:

```
1 select columnas from tabla  
2     where condicion;
```

SELECT *: Selecciona todas las columnas.

DISTINCT: Elimina duplicados en resultados.

4 Operadores Lógicos

AND, OR, NOT: Operadores booleanos.

Comparación: =, <>, <, >, <=, >=

IS NULL / IS NOT NULL: Comprueba valores nulos.

IN / NOT IN: Pertenencia a conjunto.

5 VISTAS (CREATE VIEW)

Las vistas son consultas almacenadas que pueden referenciarse como tablas.

```
1 create view nombre_vista as  
2     select ... from ... where ...;
```

6 Operaciones de Conjuntos

UNION: Unión de resultados (elimina duplicados).

```
1 select dni from programadores  
2 union  
3 select dni from analistas;
```

UNION DISTINCT: Unión eliminando duplicados explícitamente.

INTERSECT: Intersección de resultados.

```
1 select dni from programadores  
2 intersect  
3 select dni from analistas;
```

EXCEPT: Diferencia de conjuntos (A - B).

```
1 select dni from programadores  
2 except  
3 select dni from analistas;
```

EXCEPT ALL: Diferencia manteniendo duplicados.

7 JOIN - Combinación de Tablas

JOIN implícito (producto cartesiano con WHERE):

```
1 select * from tabla1, tabla2  
2     where tabla1.id = tabla2.id;
```

NATURAL JOIN: Join por columnas con mismo nombre.

```
1 select * from tabla1  
2     natural join tabla2;
```

JOIN explícito:

```
1 select * from tabla1  
2     join tabla2 on tabla1.col1 = tabla2.col2;
```

8 Funciones de Agregación

SUM(col): Suma de valores.

COUNT(*): Cuenta tuplas.

AVG(col): Media aritmética.

MAX(col): Valor máximo.

MIN(col): Valor mínimo.

```
1 select dni, sum(horas) as total  
2 from distribucion  
3 group by dni;
```

9 GROUP BY y HAVING

GROUP BY: Agrupa filas por columna(s).

```
1 select col, count(*) from tabla  
2     group by col;
```

HAVING: Filtra grupos (después de GROUP BY).

```
1 select col, count(*) from tabla  
2     group by col  
3     having count(*) > 5;
```

Diferencia: WHERE filtra antes de agrupar, HAVING después.

10 Subconsultas

Subconsulta escalar: Devuelve un único valor.

```
1 select * from tabla where col =  
2     (select max(col) from tabla);
```

Subconsulta en IN:

```
1 where dni in  
2     (select dni from analistas);
```

Subconsulta en FROM: Tabla derivada.

```
1 select * from  
2     (select dni from prog  
3     union select dni from anal);
```

11 DIVISION

Operador específico de DES para encontrar elementos que se relacionan con todos los de otro conjunto.

```
1 select dniEmp from
2 (select codigoPr,dniEmp
3  from distribucion)
4 division
5 (select codigoPr from dist
6  where dniEmp='5');
```

Interpretación: Empleados asignados a todos los proyectos del empleado '5'.

12 Operaciones Aritméticas

Se pueden realizar operaciones en SELECT:

```
1 select codigo, horas*1.2
2  from distribucion;
```

Operadores: +, -, *, /

13 Alias y RENAME

AS: Define alias para columnas/tablas.

```
1 select dni as identificador,
2  sum(horas) as total
3  from distribucion;
```

RENAME (sintaxis DES):

```
1 rename (dniEmp as dni)
2  (distribucion);
```

14 WITH - Expresiones de Tabla Comunes (CTE)

WITH ... AS permite definir tablas temporales nombradas (CTE) que simplifican consultas complejas y mejoran legibilidad. Se pueden encadenar varias CTE y usar **WITH RECURSIVE** para recursión.

Sintaxis básica:

```
1 WITH nombre_cte (columna1, ...) AS (
2  select ...
3 )
4 SELECT ... FROM nombre_cte;
```

Ejemplo 1: sumar horas por empleado y filtrar

```
1 WITH total_horas (dniEmp, total) AS (
2  SELECT dniEmp, SUM(horas) AS total
3  FROM distribución
4  GROUP BY dniEmp
5 )
6 SELECT dniEmp, total
7 FROM total_horas
8 WHERE total > 20;
```

Ejemplo 2: equivalente a DIVISION (empleados que hacen todos los proyectos de 'Evaristo')

```
1 WITH proyectos_eva (codigoPr) AS (
2  SELECT códigoPr
3  FROM distribución
4  WHERE dniEmp = (SELECT dni FROM analistas WHERE
5    nombre = 'Evaristo')
6 ),
7 empleados_conjunto (dniEmp) AS (
8  SELECT dniEmp
9  FROM distribución
10 WHERE códigoPr IN (SELECT códigoPr FROM
11   proyectos_eva)
12 GROUP BY dniEmp
13 HAVING COUNT(DISTINCT códigoPr) = (SELECT COUNT(*)
14   FROM proyectos_eva)
15 )
16 SELECT dniEmp FROM empleados_conjunto;
```

15 Uso del TOP y ORDER BY

TOP n: Limita el número de filas devueltas. **ORDER BY:** Ordena resultados por columna(s), se puede usar ASC (ascendente) o DESC (descendente).

```
1 SELECT TOP 5 dniEmp, SUM(horas) AS total
2 FROM distribución
3 GROUP BY dniEmp
4 ORDER BY total DESC;
```

16 Ejemplos Prácticos

Empleados sin teléfono:

```
1 select dni,nombre from prog
2 where telefono is null
3 union
4 select dni,nombre from anal
5 where telefono is null;
```

Suma de horas por empleado:

```
1 select dni, sum(horas) as total
2 from (select dni from prog
3  union select dni from anal)
4 join dist on dni = dniEmp
5 group by dni;
```

Proyectos sin analistas:

```
1 select codigo from proyectos
2 except
3 select distinct codigo from
4 (select codigoPr as codigo,
5  dniEmp from dist)
6 join analistas on dniEmp = dni;
```

Combinando múltiples operaciones:

```
1 create view vista_compleja as
2 (select dni,nombre,codigoPr
3  from (select * from prog
4  union select * from anal)
5  join dist on dni = dniEmp)
6 union
7 (select dni,nombre,codigo
8  from (select * from prog
9  union select * from anal)
10 join proy on dni = dniDir);
```

Combinando JOIN y WITH:

```
1 WITH pelea (barco, fecha, resultado) AS (
2  SELECT barco, fecha, resultado
3  FROM batallas join resultados
4  ON nombre = batalla
5 )
6 SELECT barco
7 FROM pelea p1 JOIN pelea p2
8  ON p1.barco = p2.barco
9 WHERE p1.fecha < p2.fecha
10  AND p1.resultado = 'victoria'
11  AND p2.resultado = 'derrota';
```

Uso de 2 JOIN simultaneos

```
1 SELECT PROJNO, DEPTNAME, MGRNO, LASTNAME
2 FROM PROJECT JOIN DEPARTMENT
3  ON PROJECT.DEPTNO = DEPARTMENT.DEPTNO
4  JOIN EMPLOYEE
5  ON DEPARTMENT.MGRNO = EMPLOYEE.EMPNO
6 WHERE DEPARTMENT.DEPTNO = 'D21'
7 ORDER BY PROJNO;
```

Que equivale a:

```
1 SELECT PROJNO, DEPTNAME, MGRNO, LASTNAME
2 FROM PROJECT P JOIN DEPARTMENT D
3  ON P.DEPTNO = D.DEPTNO
4  JOIN EMPLOYEE E
5  ON D.MGRNO = E.EMPNO
6 WHERE D.DEPTNO = 'D21'
7 ORDER BY P.PROJNO;
```

En este ultimo caso, se puede poner en el SELECT PROJNO o P.PROJNO, ya que no hay ambigüedad.